

Tracker

Webcam-based multi-track amplitude modulation player. Tracked balls control volume of audio loops via SDL2_mixer, with perspective-correction for angled camera setups.

How it works

Each ball controls two audio loops via its perspective-corrected position:

- **Ball 1 (red)**: X-axis -> Track 1 volume, Y-axis -> Track 2 volume
- **Ball 2 (green)**: X-axis -> Track 3 volume, Y-axis -> Track 4 volume
- Additional balls (when tracked) follow the same pattern: ball n -> tracks $2n$ and $2n+1$
- All tracks loop continuously, volume modulated in real-time by ball position
- If a ball is not detected (lost), its tracks go **silent** (volume -> 0)

Perspective correction

The webcam view can be angled, foreshortening ball positions. Define a planar region with 4 draggable corner handles; ball positions are rectified through a homography before mapping to volume.

- **Drag** the 4 corner handles (red circles = X axis, blue circles = Y axis) to outline the tracking plane
- **S** -- save calibration to `calib.txt` (auto-loaded next run)
- **R** -- reset corners to the full frame
- A perspective grid + quadrant crosshair is overlaid so you can see the rectification

Usage / arguments

```
./tracker [N]
```

- **N** -- number of balls to track (default 2, max 16)
- Example: `./tracker 3` tracks three balls, controlling tracks 1-6

Trackers auto-initialize at startup at staggered positions; lost trackers attempt to re-acquire.

Build (Desktop Linux)

```
make  
# or manually:
```

```
g++ main.c -o tracker -I/usr/include/opencv5 \  
$(pkg-config --cflags --libs sdl2 SDL2_mixer) \  
-lraylib -lopencv_core -lopencv_videoio -lopencv_video \  
-lopencv_imgproc -lopencv_calib -lopencv_geometry -lopencv_tracking \  
-lGL -lm -lpthread -ldl -lrt -lX11
```

Build (Android / Termux)

Zero-dependency install from a fresh Termux session:

```
# Clone the repo  
git clone <repo-url> tracker  
cd tracker  
  
# One-shot install + build (installs all deps automatically)  
bash build_termux.sh
```

Or step-by-step:

```
# 1. Enable x11-repo (for opencv, raylib, sdl2-mixer)  
pkg install -y x11-repo  
  
# 2. Install dependencies  
pkg update -y  
pkg install -y clang make pkg-config opencv raylib sdl2 sdl2-mixer  
  
# 3. Build  
make  
# or: make PLATFORM=termux
```

Termux runtime requirements

Component	Package	Purpose
Camera	termux-api	Access device camera from CLI
Display	termux-x11 or VNC	Render the window
Audio	Built-in	SDL2_mixer works out of the box

```
# Install termux-api for camera access  
pkg install termux-api  
  
# Option A: termux-x11 (recommended, better performance)  
pkg install termux-x11  
# In another session:  
termux-x11 :0 &
```

```
export DISPLAY=:0
./tracker

# Option B: VNC
pkg install tigervnc
vncserver :1 -geometry 1280x720
export DISPLAY=:1
./tracker
```

Camera on Android

Android does not expose `/dev/video0`. Use `termux-api` to pipe camera frames, or run the tracker on a device with USB webcam OTG support. OpenCV's `VideoCapture(0)` requires either: - A USB webcam via OTG (some Android devices support this) - `termux-camera-photo` piped to a virtual device (experimental)

Run

Place WAV files as `samples/track1.wav` through `samples/track8.wav` (only those needed for your ball count), then:

```
./tracker
```

Dependencies

Library	Version	Notes
Raylib	5.x-6.x	Windowing, rendering, input
OpenCV	4.x-5.x	core, video, videoio, imgproc, calib, geometry, tracking
SDL2 + SDL2_mixer	2.x	Audio playback and volume control

Controls

Input	Action
Mouse drag / Touch	Move nearest corner handle
S	Save calibration to <code>calib.txt</code>
R	Reset corners to full frame

Input	Action
-------	--------

Troubleshooting

Qt6 / opencv package fails to install on Termux

Known issue: Termux's Qt6 packages occasionally have dependency conflicts (qt6-qtbase, qt6-qtwayland, qt6-qt5compat). The opencv package depends on Qt6.

Fix (try first):

```
pkg upgrade -y
apt --fix-broken install -y
pkg install -y opencv
```

If that fails: The build_termux.sh script automatically detects this and builds OpenCV from source without Qt (~15-25 min on phone). The tracker only uses OpenCV for image processing and tracking, not GUI, so Qt is not needed.

Manual fallback:

```
pkg install -y clang make cmake git pkg-config raylib sdl2 sdl2-mixer
# Then install opencv .deb with force (skip broken Qt6 deps):
wget https://packages.termux.dev/apt/termux-x11/pool/main/o/opencv/opencv_*.deb
dpkg -i --force-depends opencv_*.deb
```

Build fails: "cannot find -lraylib"

Ensure x11-repo is enabled: `pkg install -y x11-repo && pkg install -y raylib`

Build fails: "cannot find -lGL"

Install OpenGL: `pkg install -y mesa`

Hardware / BOM (Czech)

See [HW.md](#) for the full hardware requirements and order list — 3-box setup (box A = tracker on Raspberry Pi 4, boxes B/C = wireless sampler triggers on Pi Zero 2 WH), 20 wireless buttons, 10 m mains cord, price estimate and GPIO wiring schemes. Czech language.

Files

- `HW.md` -- hardware requirements / order list (Czech)
- `main.c` -- source
- `Makefile` -- build system (desktop + Termux)
- `build_termux.sh` -- zero-dep Termux setup script
- `remote_update.sh` -- pull + rebuild on Android over SSH
- `docs/tracker_manual.pdf` -- user manual
- `calib.txt` -- saved perspective calibration (gitignored)
- `samples/` -- audio loops

Version

0.2.0